UNIVERSITY CLASSROOM ALLOCATION

A Report Submitted
in Partial Fulfilment for the Degree
of

BACHELOR OF TECHNOLOGY

in

Department of Computer Science and Engineering

by

ANIKET ROY (CSB21028)

Under the Supervision of

Prof. Utpal Sharma

Professor

Department of Computer Science and Engineering, Tezpur University

School of Engineering

Department of Computer Science and Engineering

Tezpur University

Napaam – 784028, Assam, India

June, 2025

Department of Computer Science and Engineering

Tezpur University

Napaam – 784028, Assam, India

DECLARATION

We hereby declare that the work presented in this report entitled "UNIVERSITY CLASSROOM ALLOCATION", is carried out by us. We have not submitted the matter embodied in this report for the award of any other degree or diploma of any other University or Institute. We have given due credit to the original authors/sources for all the words, ideas, diagrams, graphics, computer programs, experiments, results, that are not my original contribution. We have used quotation marks to identify verbatim sentences and given credit to the original authors/sources. We affirm that no portion of my work is plagiarized, and the experiments and results reported in the report are not manipulated. In the event of a complaint of plagiarism and the manipulation of the experiments and results, we shall be fully responsible and answerable.

Place:	Name: Suraj Subba	Name: Akash Singh Yadav	Name: Aniket Roy
Date:	Enrollment no: CSB21018	Enrollment no: CSB21023	Enrollment no: CSB21028
	Signature:	Signature:	Signature:

Department of Computer Science and Engineering

Tezpur University

Napaam – 784028, Assam, India

CERTIFICATE

This is to certify that the dissertation entitled "**UNIVERSITY CLASSROOM ALLOCATION**" submitted by **ANIKET ROY , SURAJ SUBBA , AKASH SINGH YADAV** bearing enrollment numbers **CSB21028 , CSB21018 , CSB21023** respectively has carried out there work under my supervision and guidance for partial fulfillment of the requirements and the regulations for the award of the degree of Bachelor of Technology in Computer Science & Engineering during the session 2021 – 2025 at Tezpur University. To the best of my knowledge, the matter embodied in the dissertation has not been submitted to any other university/institute for the award of any Degree or Diploma.

Date:

Utpal Sharma

Professor

Department of CSE

Department of Computer Science and Engineering

Tezpur University

Napaam – 784028, Assam, India

CERTIFICATE by HoD

This is to certify that the dissertation entitled "**UNIVERSITY CLASSROOM ALLOCATION**" is submitted by **ANIKET ROY , SURAJ SUBBA , AKASH SINGH YADAV** bearing enrollment numbers **CSB21028 , CSB21018 , CSB21023** respectively**.** They have completed their project work successfully as needed for partial fulfillment of the requirements and the regulations for the award of the degree of Bachelor of Technology in Computer Science & Engineering during the session 2021-2025 at Tezpur University. To the best of my knowledge, the matter embodied in the dissertation has not been submitted to any other university/institute for the award of any Degree or Diploma.

Head of the Department
Department of Computer Sc. & Engineering
Tezpur University

Date:

Department of Computer Science and Engineering

Tezpur University

Napaam – 784028, Assam, India

Certificate by the Examiner

This is to certify that the report entitled "**UNIVERSITY CLASSROOM ALLOCATION**" submitted by **ANIKET ROY , SURAJ SUBBA , AKASH SINGH YADAV** bearing enrollment numbers **CSB21028 , CSB21018 , CSB21023** respectively in partial fulfilment of the requirements for the degree of Bachelor of Technology in Computer Science & Engineering during the session 2021-2025 at Tezpur University has been examined by me and is found satisfactory for the award of the degree.

This approval does not necessarily endorse or accept every statement made, opinion expressed, or conclusion drawn as recorded in the report. It only signifies the acceptance of this report for the purpose for which it is submitted.

(Examiner)

Place:

Date:

ACKNOWLEDGEMENT

I take this opportunity to extend my heartfelt gratitude to Utpal Sharmah, Professor, Tezpur University for giving me the opportunity to carry out my work under him and providing me ample guidance and support through the course of the project.

I would also like to thank the Head of the Department, Dept. of Computer Science and Engineering, and all the faculty members and Staff of the dept. of CSE, Tezpur University for the valuable guidance and cooperation throughout the project.

My sincere thanks and appreciation goes to my friends and family members who have directly or indirectly helped me out with their abilities.

(Aniket Roy)

CSB21028

ABSTRACT

University classroom allocation is a fundamental task in academic administration that involves assigning courses to appropriate classrooms and time slots while satisfying a range of logistical and academic constraints. This project presents the development of a web-based system designed to automate the classroom allocation process while also offering the flexibility for manual scheduling. The objective is to ensure efficient utilization of classroom resources, avoid scheduling conflicts, and reduce the administrative burden involved in timetable creation.

The system is built using PHP and MySQL and allows administrators to input data related to courses, professors, student groups, and classrooms. A greedy algorithm processes this data to automatically allocate classrooms based on course requirements, room capacities, and availability. It ensures that no professor or student group is double-booked and that classroom capacities are respected. For cases where specific adjustments are needed, users can manually modify the generated schedule through an interactive interface, giving them complete control over final allocations.

The output is presented as a clear, tabular timetable that maps days and time slots to courses and assigned classrooms. This dual approach—automated generation combined with manual flexibility—enhances usability and accommodates a wider range of institutional needs. The system is particularly useful for academic planners seeking to streamline scheduling while retaining the ability to make custom changes as necessary.

This project illustrates the practical integration of algorithmic logic and user-driven control in solving complex scheduling problems in educational institutions. Future improvements may include enhanced conflict detection during manual edits and support for more advanced optimization goals.

LIST OF TABLES

	Page No.
4.6.1 Classroom Table	17
4.6.2 Courses Table	17,18
4.6.3 Professor Table	18
4.6.4 Admin Table	18
4.6.5 Schedule Table	18,19
4.6.6 Building/Department Table	19
5.2 Technology Stack	21
6.2 Test Cases and Results	29
6.3 Performance Evaluation	30

LIST OF FIGURES

Page No.

4.2.1 Use Case Diagram for Admin 11
4.2.2 Use Case Diagram for Professor 12
4.3.1 Sequence Diagram for Admin and Professor 12
4.3.2 Sequence diagram to add function like classroom, professor, courses etc 13
4.3.3 Sequence Diagram to delete function like classroom, professor, courses etc 13
4.4 ER Diagram 14
4.5.1 Level 0 Data Flow Diagram for Admin 15
4.5.2 Level 0 Data Flow Diagram for Professor 15
4.5.3 Level 1 Data Flow Diagram for Admin 16
4.5.4 Level 2 Data Flow Diagram 16

5.6.1 Admin Dashboard page 23
5.6.2 Course Management page (Adding, Updating, Deleting & Viewing Course) 24
5.6.3 Final generated Academic Timetable page 24, 25
5.6.4 Professor Dashboard page 25
5.6.5 Excel Upload Page 26
5.6.6 PDF Export Button 26
5.6.7 Professor Timetable 27

TABLE OF CONTENTS

	Page No.
Declaration	ii
Certificate by Supervisor	iii
Certificate by HoD	iv
Certificate by the Examiner	v
Acknowledgement	vi
Abstract	vii
List of Tables	viii
List of Figures	ix
Table of Contents	x,xii
CHAPTER 1: INTRODUCTION	**1-3**
1.1 Problem Statement	1
1.2 Motivation	2
1.3 Objective	2
1.4 Scope	2,3
1.5 Methodology	3
CHAPTER 2: LITERATURE REVIEW	**4 - 5**
2.1 Existing System & Limitation	4
2.2 Automated Allocation System	4
2.3 Related Web-Based System	4
2.4 Existing Gaps and Justification for the Proposed System	5
2.5 Summary -	5
CHAPTER 3: SYSTEM ANALYSIS AND REQUIREMENTS	**6 - 9**
3.1 Requirement Gathering	6
3.2 Feasibility Study	6 , 7
3.2.1 Technical Feasibility	6
3.2.2 Operational Feasibility	6
3.2.3 Economic Feasibility	7
3.3 Existing System	7
3.4 Proposed System Overview	7
3.5 Functional Requirements	7,8
3.6 Non-Functional Requirements	8
3.7 System Constraints	8
3.8 Scheduling Constraints	8 , 9
3.8.1 Hard Constraints	8
3.8.2 Soft Constraints	9

CHAPTER 4: SYSTEM DESIGN **10 - 20**

4.1 System Architecture 10
4.1.1 Presentation Layer (Front End 10
4.1.2 Business Logic Layer (Back End) 10
4.1.3 Data Layer (Database) 10
4.2 Use Case Diagram 11 , 12
4.3 Sequence Diagram 12 , 13
4.4 Entity Relationship Diagram 14
4.5 Data Flow Diagrams 15 , 16
4.6 Database Schema 17 , 19
4.6.1 Classroom Table - 17
4.6.2 Courses Table 17 , 18
4.6.3 Professors Table 18
4.6.4 Admin Table 18
4.6.5 Schedule Table 18 , 19
4.6.6 Building/Department Table 19

4.7 Module Description 19 , 20
4.7.1 Admin Module - 19
4.7.2 Professor Module 19
4.7.3 Timetable Generation Module 19 , 20

4.8 User Interface Design 20

CHAPTER 5 IMPLEMENTATION **21 - 27**

5.1 Development Environment 21
5.2 Technology Stack 21
5.3 Database Implementation 21 , 22
5.4 Backend Logic and Functionality 22
5.5 Timetable Generation Logic 22 , 23
5.6 Screenshot of Key Modules 23
5.6.1 Admin Dashboard 23
5.6.2 Add Course 24
5.6.3 Timetable View 24 , 25
5.6.4 Professor Portal 25
5.6.5 Excell upload page 26
5.6.6 PDF Export Button 26
5.6.7 Professor Timetable 27

CHAPTER 6: UNIT TESTING AND RESULTS **28 - 30**
6.1 Testing Strategies 28
6.1.1 Unit Testing 28
6.1.2 Integration Testing 28
6.1.3 System Testing 28, 29
6.1.4 User Acceptance Testing (UAT) 29
6.2 Test Cases and Results 29
6.3 Performance Evaluation 30
6.4 Limitations Observed 30

CHAPTER 7: CONCLUSION AND FUTURE WORK **31 - 32**
7.1 Conclusion 31
7.2 Summary of Achievements 31, 32
7.3 Future Work 32

References **33**

CHAPTER 1: INTRODUCTION

In many universities, classroom allocation is still managed manually or through semi-automated processes that are inefficient, error-prone, and time-consuming. As the number of students, courses, and faculty increases, administrators are faced with the challenge of ensuring that classrooms are properly assigned based on availability, room capacity, professor schedules, and student group conflicts. Overlapping class timings, underutilized classrooms, and last-minute changes further complicate the task.

There is a critical need for a system that can streamline the allocation process by automatically assigning classrooms to courses while satisfying a range of academic and logistical constraints. At the same time, the system must offer flexibility by allowing manual adjustments when unique scheduling requirements arise. The lack of such a comprehensive solution often results in administrative delays, student dissatisfaction, and inefficient use of institutional resources.

This project addresses these challenges by developing a web-based classroom allocation system that combines the power of automated scheduling with the adaptability of manual input. The goal is to enhance accuracy, improve classroom utilization, and reduce the workload of academic staff involved in timetable planning.

1.1 Problem Statement

Manual classroom allocation in universities often results in a number of recurring problems. Despite best efforts, administrators may encounter overlapping room schedules, classrooms assigned beyond their capacity, or even overlooked constraints such as the need for technological equipment. These challenges stem from the inherent limitations of manual systems, especially when dealing with:

- Multiple academic departments with varied scheduling needs.
- Limited classroom availability and specialized room requirements.
- Diverse course loads requiring different time and space allocations.
- Lack of real-time visibility into room usage and availability.

The absence of a centralized system often forces department heads to work in silos, which leads to miscommunication and scheduling conflicts. Additionally, changes to schedules — such as newly introduced courses or faculty unavailability — are difficult to accommodate in a static, manually maintained system.

1.2 Motivation

Current challenges in university scheduling include:

- **Time-Consuming Processes**: Manual allocation takes weeks and requires repeated revisions.
- **Scheduling Conflicts:** Overlaps in professor/student group assignments or room bookings.
- **Inflexibility:** Rigid systems cannot adapt to last-minute changes (e.g., room maintenance, faculty unavailability).
- **Inefficient Resource Use:** Classrooms may remain underutilized due to poor allocation strategies.

This project addresses these issues by combining algorithmic automation (for speed and conflict avoidance) with manual override capabilities (for adaptability), ensuring a practical balance between efficiency and control. Such a system would be highly beneficial not only for administrators but also for Professors who can access their personalized timetables online.

1.4 Objectives

The primary objective of this project is to design and implement a university classroom allocation system that efficiently assigns classrooms to courses based on various constraints such as time, room capacity, and availability. The system also aims to:

- Automate the allocation of classrooms using a greedy algorithm.
- Prevent scheduling conflicts for professors and student groups.
- Maximize classroom occupancy while respecting constraints (e.g., room size, equipment needs).
- Allow manual adjustments to the generated timetable when needed.
- Present the final schedule in a user-friendly, tabular format.

1.5 Scope

This project is aimed at addressing the challenges of classroom allocation and timetable generation in a university environment. It is designed to automate the scheduling process while considering key academic and logistical constraints. The system focuses on the efficient allocation of classrooms and time slots based on predefined data inputs.

Key functionalities include:

- An administrative interface to manage classrooms, buildings, professors, and courses.
- A scheduling engine that automatically generates conflict-free timetables based on lecture and tutorial hours.

- Role-specific dashboards for professors to access their personalized timetables.
- Enforcement of classroom-specific constraints such as seating capacity, projector availability.
- Support for the L-T-P (Lecture-Tutorial-Practical) structure, using only Lecture and Tutorial hours for scheduling. Practical hours are acknowledged but excluded from this version.

Limitations of the current system:

- Management of student course Enrollment or registration processes; it assumes all course allocation data is preloaded.
- It does not handle scheduling for backlog students, i.e., students reattempting courses from previous semesters.
- The system does not account for situations where students from different departments choose the same open elective.

1.6 Methodology

The development of the University Classroom Allocation System follows a structured and modular approach. The Waterfall Model of the Software Development Life Cycle (SDLC) has been used due to its clarity and suitability for academic systems. This model ensures that each phase—requirements analysis, system design, implementation, testing, deployment, and maintenance—is completed sequentially, allowing systematic development and easier tracking of progress.

CHAPTER 2: LITERATURE REVIEW

Classroom and timetable allocation is a critical administrative task in educational institutions. Efficient allocation improves resource utilization, reduces conflicts, and ensures a smooth academic schedule. Numerous studies and systems have been proposed to solve this problem using different methodologies. This chapter explores key literature related to classroom allocation, timetable scheduling, and the technologies used, laying the foundation for the design choices made in the current project.

2.1 Existing System and Limitations

Historically, classroom allocation has been managed manually or with basic spreadsheet tools. While simple, these methods are prone to human error and become unmanageable as the institution scales. Manual methods cannot easily adapt to sudden changes in faculty availability, room maintenance schedules, or student batch sizes. Additionally, they lack the ability to validate constraints like classroom capacity, projector availability, or overlapping class timings.

2.2 Automated Allocation Systems

Allocation systems gained popularity as educational institutions grew. Various algorithms like Genetic Algorithms, Constraint Satisfaction Problems (CSP), and Integer Linear Programming have been used to solve these problems.

For example, Boomija et al. (2019) proposed a Smart and Dynamic Timetable Generator that uses optimization algorithms to allocate slots effectively while considering constraints. Similarly, Abhinaya et al. (2019) developed an Online Application of Automatic Timetable Generator that streamlines scheduling but focuses more on timetable creation than classroom allocation.

However, many of these systems emphasize overall timetable optimization without dedicating sufficient focus to room-level constraints, such as whether the room has a projector or its seating capacity is suitable for the batch.

2.3 Related Web-Based Systems

Web-based systems have emerged as a practical solution for dynamic allocation. They offer role-based access (e.g., admin, department), scalability, and central data management. Some recent works use web technologies like PHP, MySQL, and JavaScript to build interfaces that allow non-technical users to manage academic resources more efficiently.
In most cases, however, these systems focus on timetable generation alone and assume that classrooms are available and suitable. The challenge of ensuring classroom suitability (capacity, projector, availability) is often overlooked or delegated to manual checks.

2.4 Existing Gaps and Justification for the Proposed System

- Despite advancements, existing solutions have several gaps:
- Lack of projector-based filtering
- No mechanism to link room allocation with departmental requirements
- Difficulty managing large institutions with multiple departments
- User interfaces are often not intuitive for admin users
- The proposed University Classroom Allocation system addresses these issues directly:
- It includes multi-department support with dynamic filtering of classrooms based on projector and capacity constraints.
- The admin dashboard is user-friendly and allows easy management of buildings, rooms, and courses.
- The system uses greedy scheduling for fast, real-time allocation instead of complex algorithms that may be slower or harder to maintain.
- It provides modularity with different PHP files for adding/editing data, improving maintainability.

2.5 Summary

The literature shows that while many systems have aimed to automate academic scheduling, few offer a comprehensive solution tailored to classroom constraints. This project bridges that gap by focusing on room-level attributes and administrative usability. It combines the strengths of existing systems with practical features that make it suitable for universities aiming to modernize their allocation process.

CHAPTER 3: SYSTEM ANALYSIS AND REQUIREMENTS

3.1 Requirement Gathering

The requirement gathering phase involved understanding the operational needs of a typical university's academic administration. The key stakeholders identified in this process were:

- **Admin**: Responsible for managing buildings, classrooms, courses, faculty assignments, and timetable generation.
- **Professors**: Expected to log in, manage their assigned courses, and view their personalized teaching schedule.

The analysis revealed that a centralized, automated system would significantly enhance the efficiency of classroom and timetable allocation processes.

3.2 Feasibility Study

A feasibility study was conducted to determine whether the proposed system could be implemented effectively in a real-world academic environment. This study included an evaluation of technical, operational, and economic feasibility.

3.2.1 Technical Feasibility

The system is developed using widely adopted open-source technologies:

- **Frontend**: HTML, CSS, JavaScript
- **Backend**: PHP
- **Database**: MySQL
- **Server Environment**: Apache (XAMPP)

These technologies are compatible with most academic server environments and require minimal resources to deploy and maintain, making the system technically feasible.

3.2.2 Operational Feasibility

Designed with simplicity and usability in mind, the system ensures:

- A clean and intuitive interface for administrative users.
- Minimal training requirements for adoption.
- Secure login and role-based access for professors to manage their schedules.

Automating the manual scheduling process will reduce errors, save time, and improve transparency and efficiency.

3.2.3 Economic Feasibility

Since the system uses entirely open-source tools, there are no licensing or software costs. The project was developed as part of an academic exercise, eliminating labor and development expenses. Institutions only need minimal server infrastructure, which makes the solution cost-effective and economically viable.

3.3 Existing System

In most universities, classroom and timetable allocations are performed manually or through spreadsheets. These conventional methods exhibit several limitations:

- High risk of **scheduling conflicts**.
- **Manual updates** are time-consuming and inefficient.
- Lack of **real-time access** to updated timetables.
- Inability to **scale** for growing student populations or multiple departments.
- No support for **classroom constraints** such as capacity or projector availability.

3.4 Proposed System Overview

The proposed system aims to automate the classroom and timetable allocation process with respect to course credits, available resources, and faculty assignments. It provides a robust, centralized interface with the following core features:

- **Admin Panel**: Manage buildings, classrooms, courses, professors, and allocations.
- **Automated Timetable Generation**: A greedy scheduling approach is used for fast allocation.
- **Constraint Satisfaction**: Prevents conflicts in time slots, room usage, and faculty schedules.
- **Role-Based Access**: Admin and professor roles with different access privileges.
- **Personalized Schedules**: Professors can view schedules tailored to their assigned courses.

3.5 Functional Requirements

i. Administrator Capabilities:
   - Add, edit, or delete:
     - Courses
     - Classrooms
     - Buildings
     - Professors
   - Assign courses to professors and semesters.
   - Generate and manage the academic timetable.

ii. Professor Capabilities:
   - Log in securely.
   - Add or update their course details.
   - View their personalized teaching schedule.

iii. Timetable Generation Requirements:
   - Must adhere to **L+T hours** (Lecture + Tutorial) defined by course credits.
   - Prevent **time-slot conflicts** for professors or semesters.
   - Allocate classrooms based on **availability**, **capacity**, and **required features**.

3.6 Non-Functional Requirements

- **Performance**: The system should generate timetables within a few seconds for all departments.
- **Scalability**: Should allow for future integration with elective registration or student Enrollment systems.
- **Usability**: The interface should be responsive, clean, and intuitive for all user types.
- **Security**: Role-based access and password-protected login system to protect user data.
- **Reliability**: Must ensure generation of consistent, conflict-free timetables.

3.7 System Constraints

While the system improves significantly on manual processes, it currently has the following limitations:

- Does not support scheduling for **backlog students** or **repeat course registrations**.
- **Open-elective conflicts** across departments are not resolved automatically.
- Only supports **predefined semesters** (Semester I to Semester VIII).
- Does not currently integrate with full **student information systems**.
- **Elective course conflicts** are not yet resolved dynamically.

3.8 Scheduling Constraints

3.8.1 Hard Constraints

These are mandatory rules that must be strictly followed:

- A professor cannot be scheduled for more than one class at the same time.
- A classroom cannot host more than one class at the same time.
- Courses must match their specified **Lecture (L) + Tutorial (T)** hours.
- Courses can only be assigned to their designated **semester**.
- Classrooms must meet the required **seating capacity**.
- Professors can only be scheduled for **courses they are assigned**.
- Classes must be conducted **only in available classrooms**.
- Class timings must fall within **official university hours** (e.g., 9:00 AM – 5:00 PM).
- **Practical sessions (P)** are currently not part of the scheduling logic.

3.8.2 Soft Constraints

These rules are preferable but not mandatory and may be relaxed:

- Avoid scheduling **more than two consecutive classes** for any professor.
- Distribute lectures **evenly across the week** for each semester.
- Prefer assigning **projector-enabled classrooms** for courses that require them.
- Assign the **same classroom to a batch for the entire week**, if available.

CHAPTER 4: SYSTEM DESIGN

4.1 System Architecture

The University Classroom Allocation System adopts a **three-tier architecture**, ensuring modularity, maintainability, and scalability. The three main layers are:

4.1.1 Presentation Layer (Frontend)

This layer is responsible for user interaction and interface design. It includes:

- Technologies Used: HTML, CSS, and JavaScript
- Features:
  - Forms for course, professor, and classroom management
  - Login pages for admin and professors
  - Timetable views

4.1.2 Business Logic Layer (Backend)

This is the core processing layer developed using PHP. It manages:

- User authentication and session handling
- CRUD operations (Create, Read, Update, Delete)
- Timetable generation logic using a greedy scheduling algorithm
- Validation of constraints and allocation rules

4.1.3 Data Layer (Database)

The backend communicates with a **MySQL** database to perform data operations. It stores:

- Buildings, classrooms, and their capacities
- Courses with L-T-P credit structure
- Professors and their assigned courses
- Semesters, time slots, and scheduled classes
- Login credentials with role-based access control

4.2 Use Case Diagram

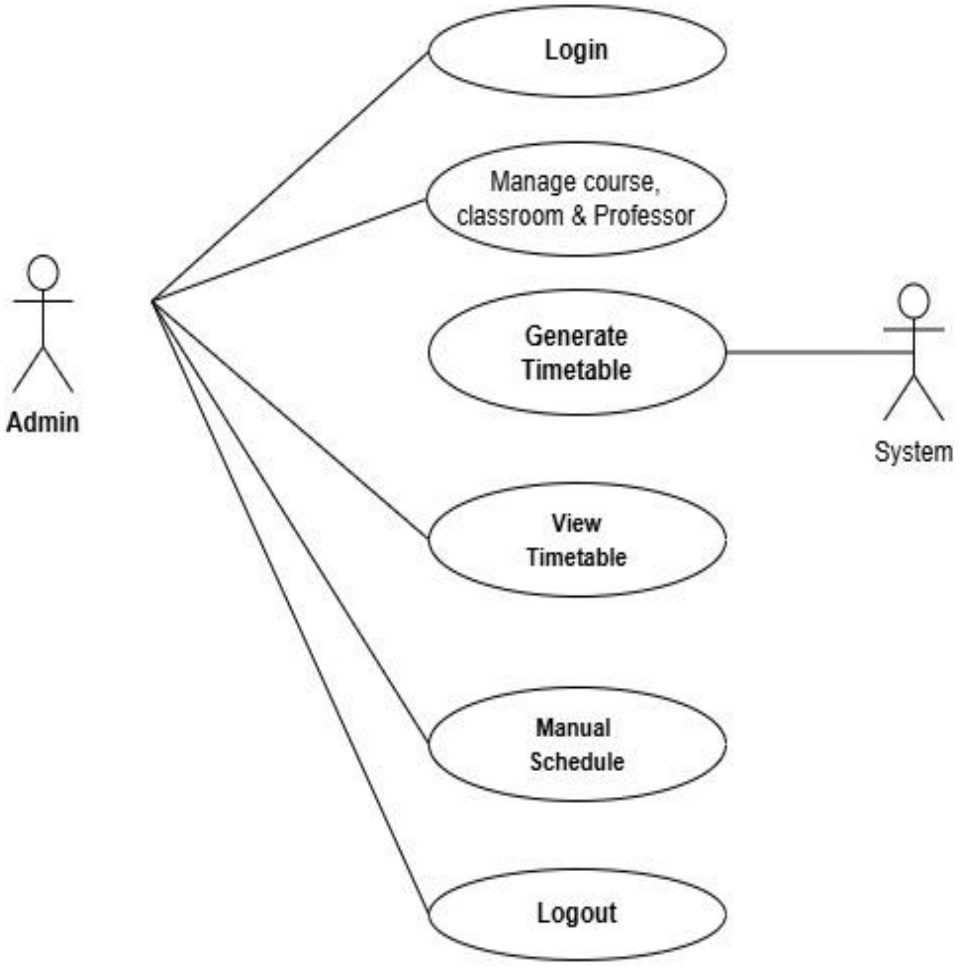


Fig 4.2.1 Use Case Diagram for Admin

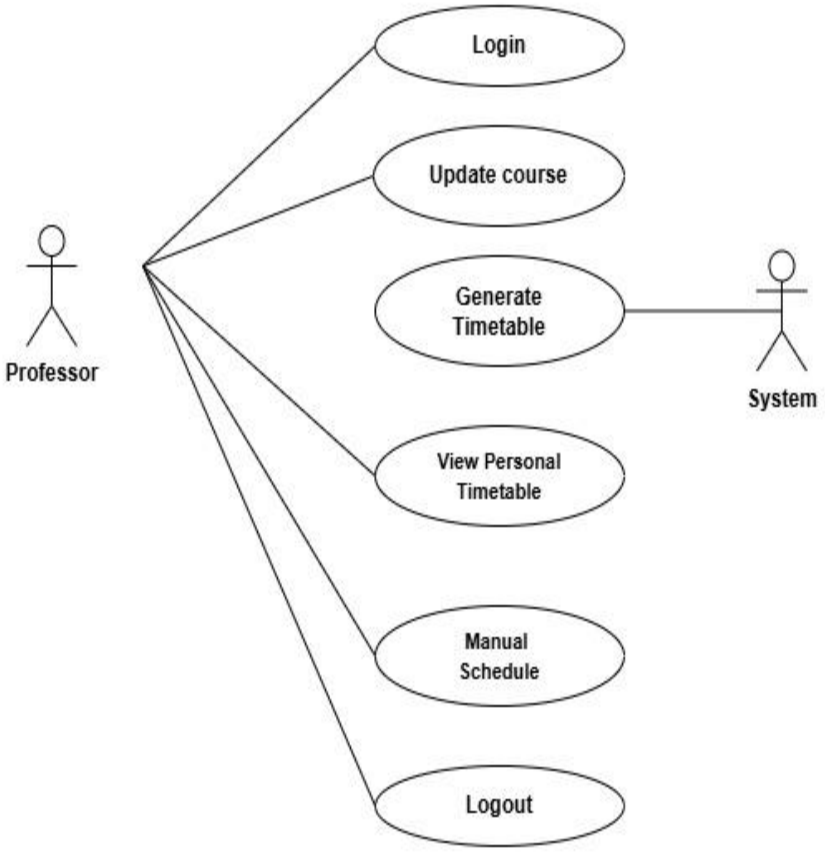


fig 4.2.2 Use Case Diagram for Professor

4.3 Sequence Diagram

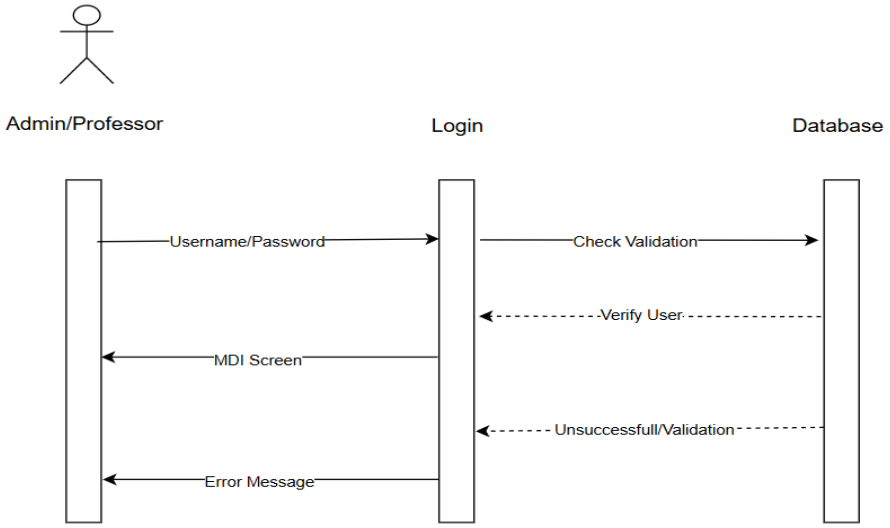


Fig 4.3.1 Sequence diagram for Admin and Professor Login

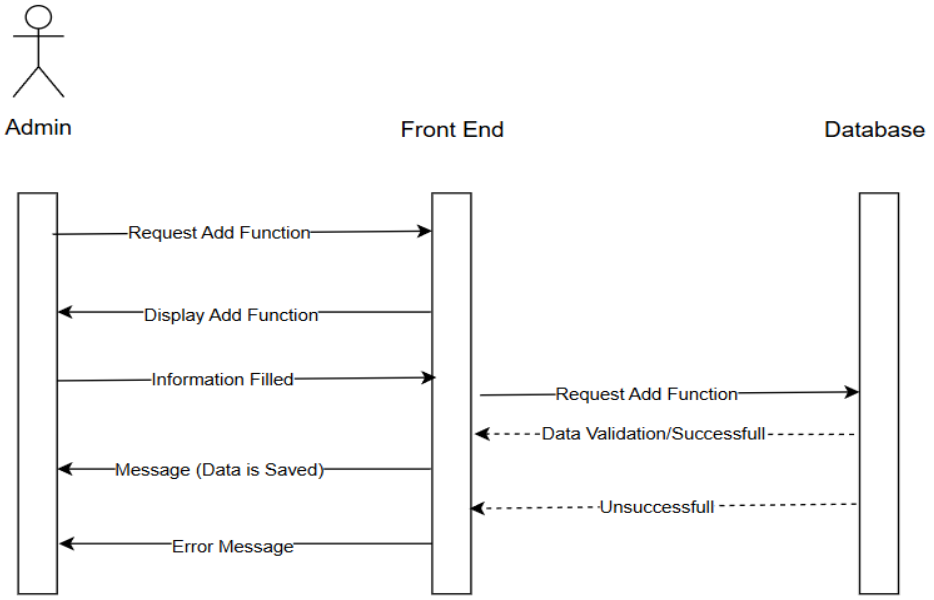


Fig 4.3.2 Sequence diagram to add function like classroom, professor, courses etc.

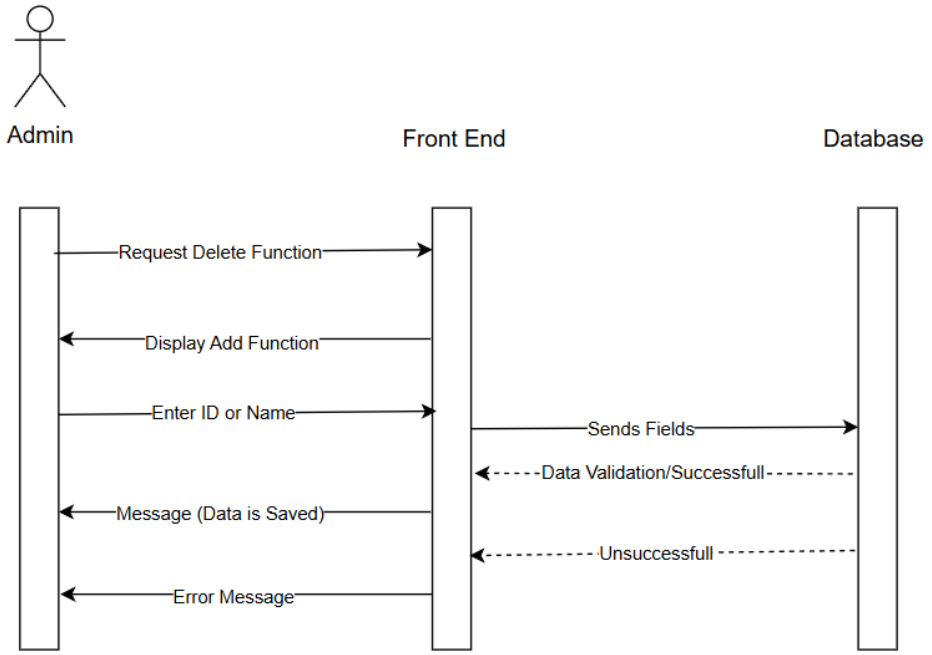


Fig 4.3.3 Sequence Diagram to delete function like classroom, professor, courses etc

4.4 Entity-Relationship Diagram

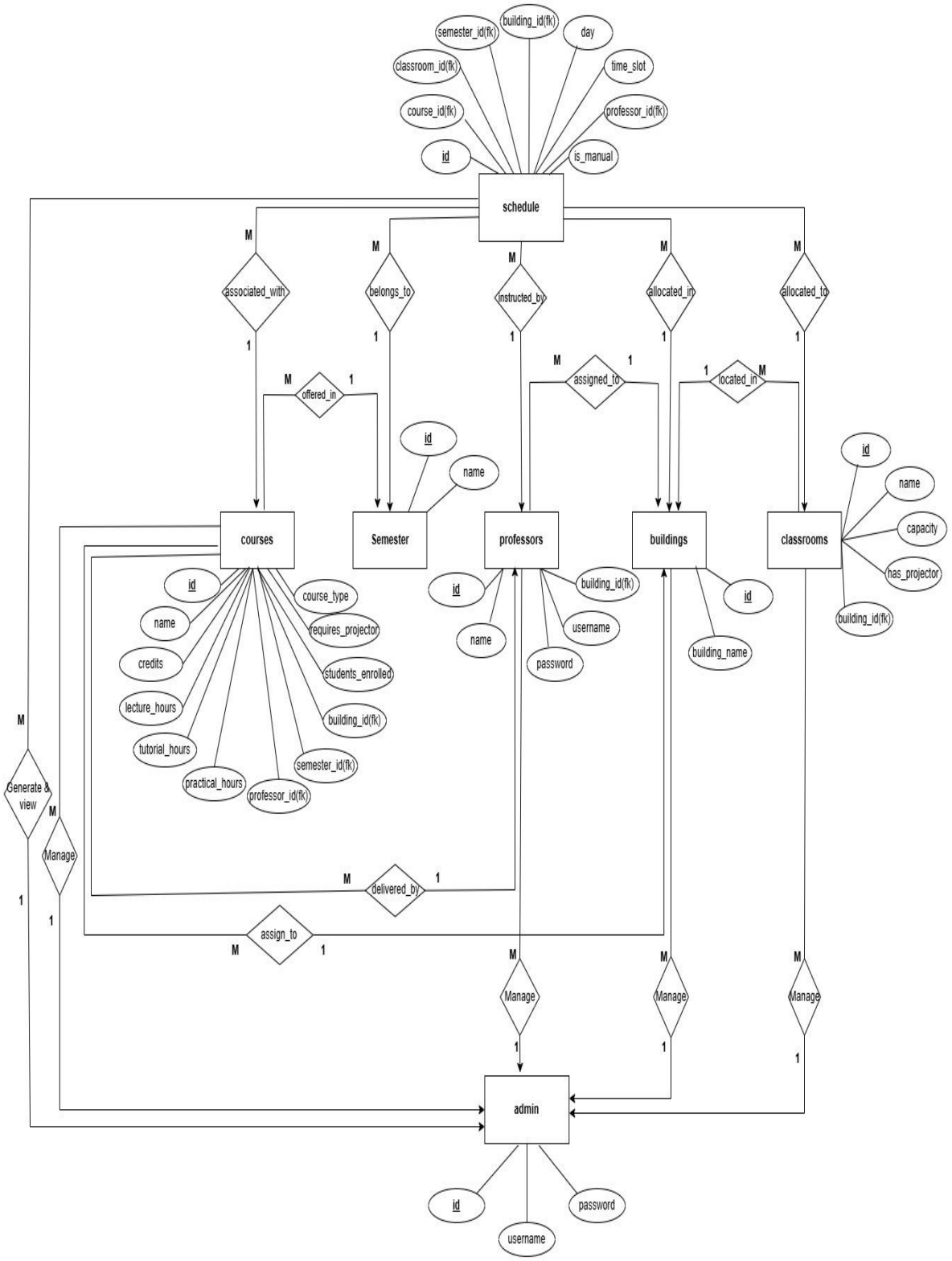


Fig 4.4 Entity Relationship Diagram

4.5 Data Flow Diagram

LEVEL 0 DFD

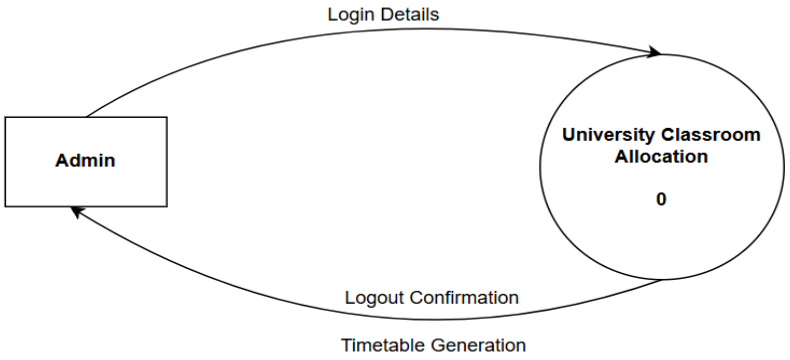


Fig 4.5.1 Level 0 Data Flow Diagram for Admin

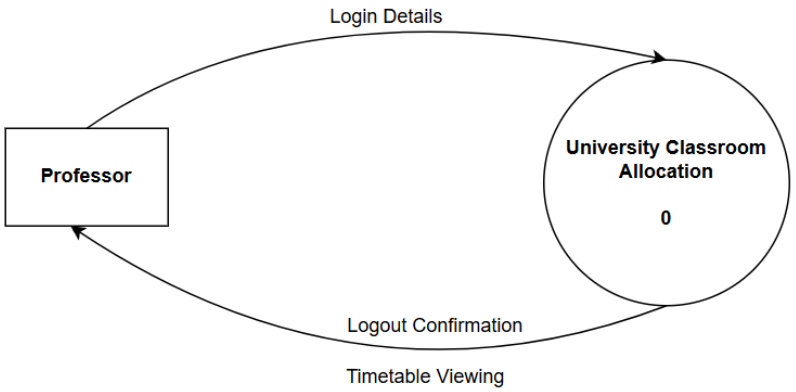


Fig 4.5.2 Level 0 Data Flow Diagram for Professor

LEVEL 1 DFD

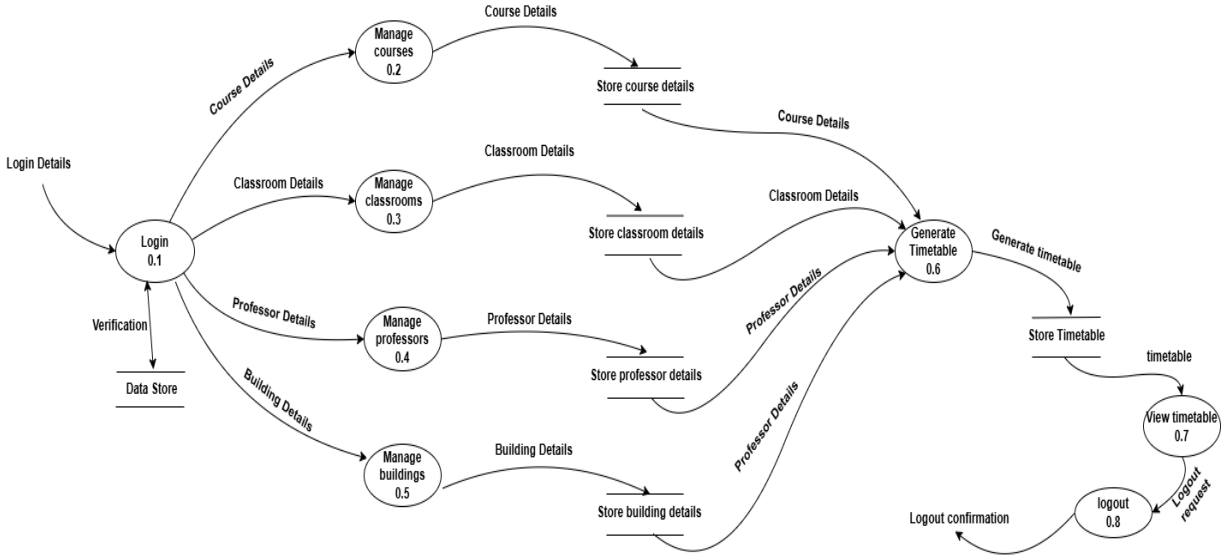


Fig 4.5.3 Level 1 Data Flow Diagram for Admin

LEVEL 2 DFD

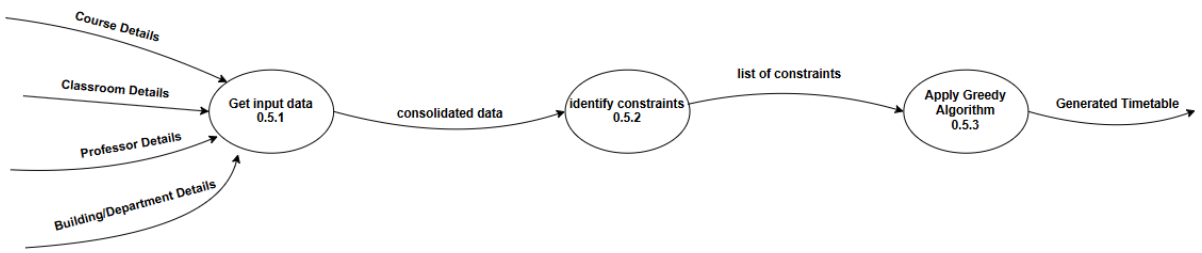


Fig 4.5.4 Level 2 Data Flow Diagram

4.6 Database Schema

The database is relational and normalized to reduce redundancy. Below are the core tables:

4.6.1 Classroom Table

Field Name	Data Type	Description
id	int	Unique ID for the classroom
name	varchar	Name of the classroom
capacity	int	Maximum number of students
has_projector	boolean	Whether room has a projector
building_id	int	Unique ID for the building

4.6.2 Courses Table

Field Name	Data Type	Description
id	int	Unique identifier for the course
name	varchar	Name of the course
credits	int	Total credits of the course
lecture_hours	int	Total hours of lecture
tutorial_hours	int	Total hours of tutorial
practical_hours	int	Total hours of practical
professor_id	int	Unique identifier for the professor
semester_id	int	Unique identifier for the semester
building_id	int	Unique identifier for the building

students_enrolled	int	Total no. of students enrolled for that course
requires_projector	boolean	Whether the course needs a projector
course_type	varchar	Type of that course (core/elective/open elective)

4.6.3 Professors Table

Field Name	Data Type	Description
id	int	Unique identifier for the professor
name	varchar	Full name of the professor
building_id	int	Unique identifier for the building
username	varchar	Unique username for the professor to login
password	varchar	Unique password for the professor to login

4.6.4 Admin Table

Field Name	Data Type	Description
id	int	Unique identifier for the admin
username	varchar	Unique username for the admin to login
password	varchar	Unique password for the password to login

4.6.5 Schedule Table

Field Type	Data Type	Description
id	int	Unique id for each course after scheduling
course_id	int	Unique id for an particular course
classroom_id	int	Unique id for an particular classroom

semester_id	int	Unique id for an particular semester
building_id	int	Unique id for an particular building/department
day	varchar	A particular day of the week
time_slot	int (range)	Range of time for an course
professor_id	int	Unique id for an professor
is_manual	boolean	Looks if the course is manually scheduled or not.

4.6.6 Building/Department Table

Field Type	**Data Type**	**Description**
id	int	Unique id for a building/department
building_name	varchar	Name of the building

4.7 Module Description

4.7.1 Admin Module

- Add/Edit/Delete:
  - Classrooms, courses, buildings, professors
- Assign professors to courses
- Import data from Excel sheets
- Generate timetable automatically
- Export timetable as PDF
- View the complete university-wide timetable

4.7.2 Professor Module

- Login securely with credentials
- Add or modify course details
- View a personalized weekly schedule based on assigned courses

4.7.3 Timetable Generation Module

- Uses a **Greedy Algorithm** to schedule classes
- **Factors Considered**:
  - L+T hours for each course

    - Availability of professors and classrooms
    - Room capacity and projector availability
    - No overlapping sessions
- **Hard Constraints** are strictly enforced (e.g., no double-booking)
- **Soft Constraints** are followed when possible to improve quality:
    - Try to schedule projector-required classes in equipped rooms
    - Prefer using the same room for a batch throughout the week
- The final output is saved in the schedule table and displayed on the dashboard or downloadable as PDF

4.8 User Interface Design

The UI is designed with a focus on clarity and ease of use. Key features include:

- **Responsive layouts** compatible with desktops and tablets
- **Navigation bar** with quick access to all functionalities
- **Forms** for data entry with dropdowns, validation, and feedback
- **Table views** for listing courses, classrooms, schedules
- **Login interface** for role-based access
- **Dashboard cards** showing statistics like total classrooms, professors, and scheduled slots

Screens have minimal clutter and consistent styling to ensure ease of navigation for both admins and professors.

CHAPTER 5: IMPLEMENTATION

5.1 Development Environment

The system was developed and tested in a local environment using commonly available open-source tools and platforms. The detailed development environment is as follows:

- **Operating System**: Windows 10
- **Web Server**: XAMPP (Apache, MySQL, PHP 8.x)
- **Code Editor**: Visual Studio Code
- **Web Browsers**: Google Chrome, Mozilla Firefox
- **Database Tool**: phpMyAdmin (accessed via XAMPP)

The project files were hosted in the htdocs directory of the XAMPP server, allowing for smooth local development, testing, and demonstration.

5.2 Technology Stack

Layer	Technology Used
Frontend	HTML5, CSS3, Bootstrap, JavaScript
Backend	PHP
Database	MySQL
Scheduling Logic	PHP (custom Greedy Algorithm)
PDF Export	FPDF (PHP PDF generation library)
Excel Import	CSV file parsing using PHP

These technologies were selected due to their open-source nature, ease of deployment, and wide use in academic projects. The stack allows flexible modular development while ensuring compatibility across systems.

5.3 Database Implementation

The system uses a MySQL database named **university_timetable**. The schema is normalized to ensure data consistency, minimize redundancy, and allow modular feature expansion.

Core Tables:

- *admin* – stores admin login credentials
- *professors* – stores professor data including department
- *buildings* – stores information on university buildings
- *classrooms* – includes room capacity, building, and projector availability
- *courses* – course names, L-T-P values, type, and semester assignment

- *schedule* – stores generated timetable with course, professor, classroom, and time
- *semesters* – predefined semesters (I–VIII)
- *users* – stores role-based login credentials

Each table uses appropriate **primary and foreign keys** for referential integrity. AUTO_INCREMENT is used for unique identification of records. Data types are optimized, and indexing is applied to key fields for performance.

5.4 Backend Logic and Functionality

The PHP-based backend handles all business logic through modular scripts. Key components include:

- **Role-Based Authentication**: Separate login interfaces for Admin and Professor with secure session management.
- **CRUD Operations**: Admin can perform create, read, update, and delete operations for:
  - Courses, Classrooms, Professors, Buildings
  - Assigning courses to professors and semesters
- **Excel Upload**: Admin can upload `.csv` files to bulk import course, professor, and classroom data. Uploaded files are parsed using PHP and inserted into respective tables after validation.
- **PDF Export**: Final generated timetables can be exported using the **FPDF** library.

All form submissions are validated and sanitized to protect against SQL injection. Backend logic is structured into reusable PHP functions and integrated with the MySQL database using SQL queries.

5.5 Timetable Generation Logic (Greedy Algorithm)

The timetable generation is implemented using a **Greedy Algorithm** approach. It ensures efficient allocation of classrooms and time slots based on course structure and constraints.

Algorithm Workflow**:**

i. **For each course**, the system:
   - Calculates the total sessions needed based on L (lecture) + T (tutorial) credits.

ii. **Slot Allocation**:
   - Iteratively assigns available time slots from a predefined table (`time_slots`), ensuring no conflicts with:
     - **Other courses in the same semester**
     - **Professor schedules**
     - **Room occupancy**

iii. **Constraints Considered**:

   - **Hard Constraints** (Strictly followed):

     - No overlapping classes for a professor
     - No multiple bookings of the same classroom

- Total hours = L + T per course
- Correct semester-course mapping
- Room capacity must be sufficient
- Working hours: 9:00 AM to 5:00 PM

- **Soft Constraints** (Preferred, but flexible):

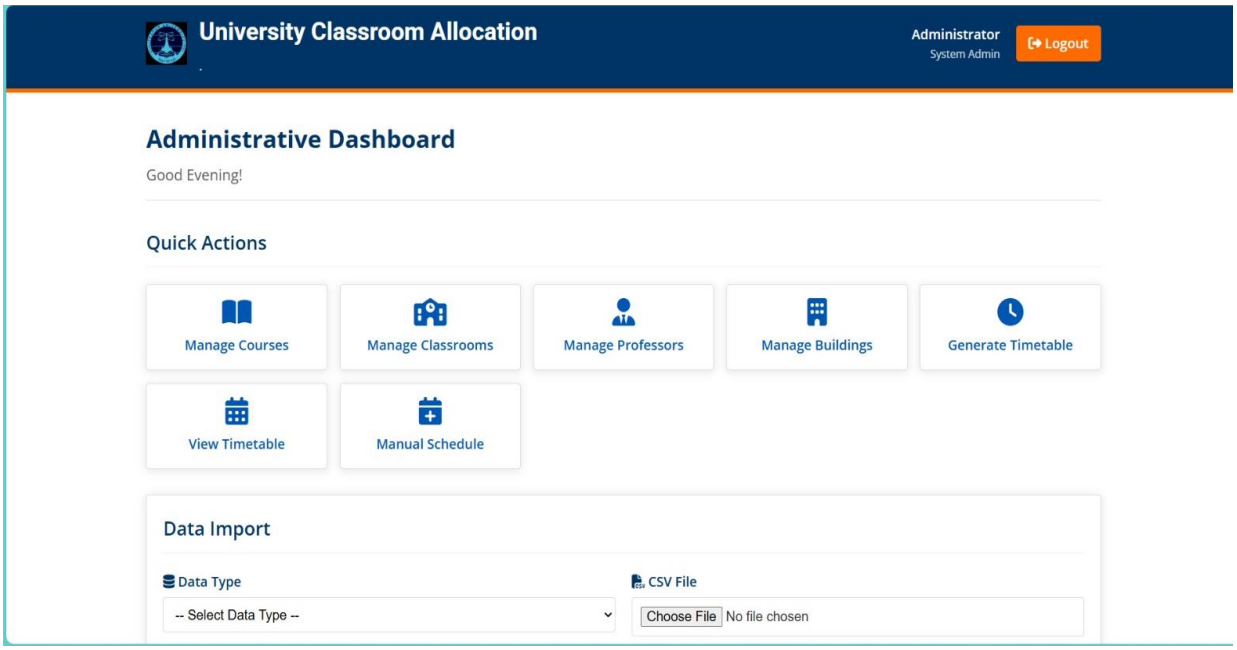
  - Prefer assigning the same classroom to a batch throughout the week
  - Try to avoid more than two back-to-back classes for professors
  - Distribute sessions across the week
  - Assign projector-enabled rooms for courses needing one

iv. **Output**:

- The finalized timetable is stored in the `schedule` table
- It includes mapped course, professor, building, classroom, and time slot
- Can be viewed online or downloaded as a PDF

5.6 Screenshots of Key Modules

5.6.1 Admin Dashboard



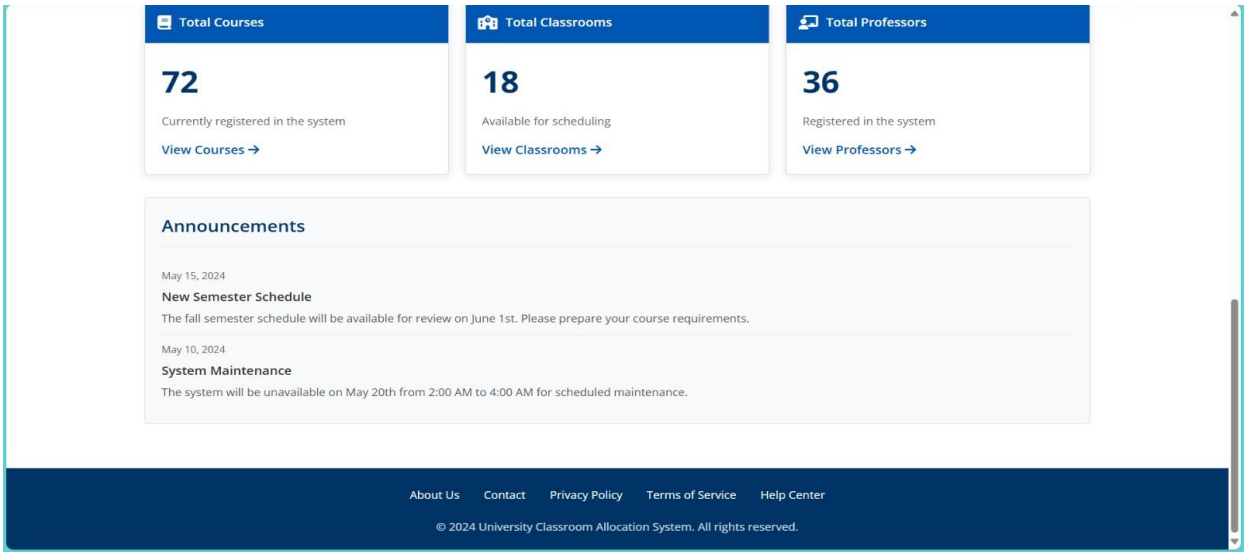


Fig 5.6.1 Admin main Dashboard page

5.6.2 Add Course Form

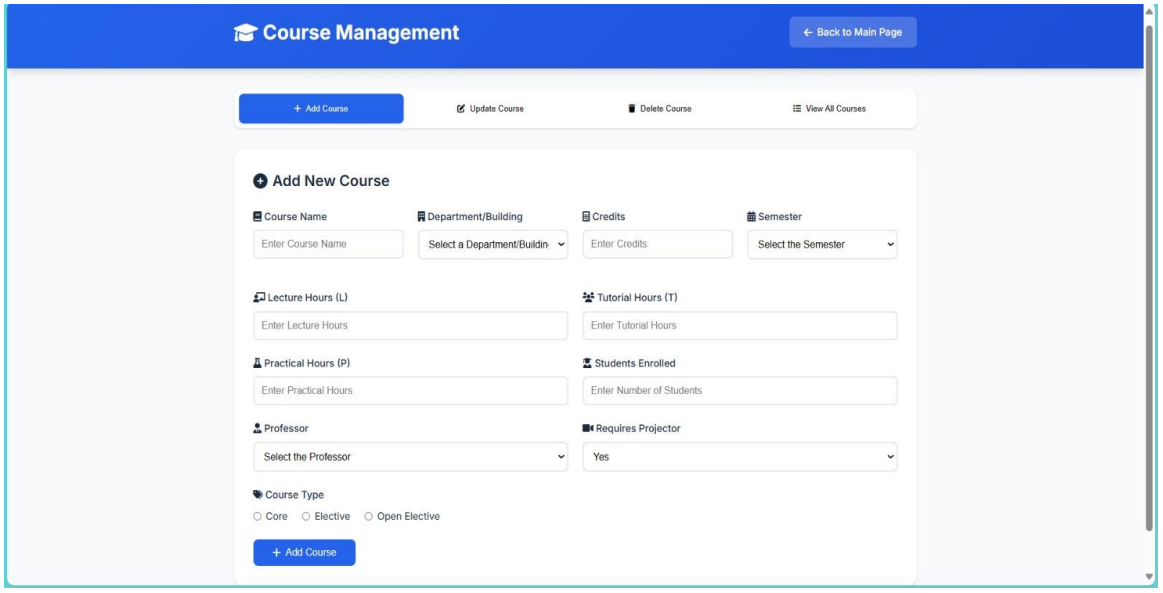
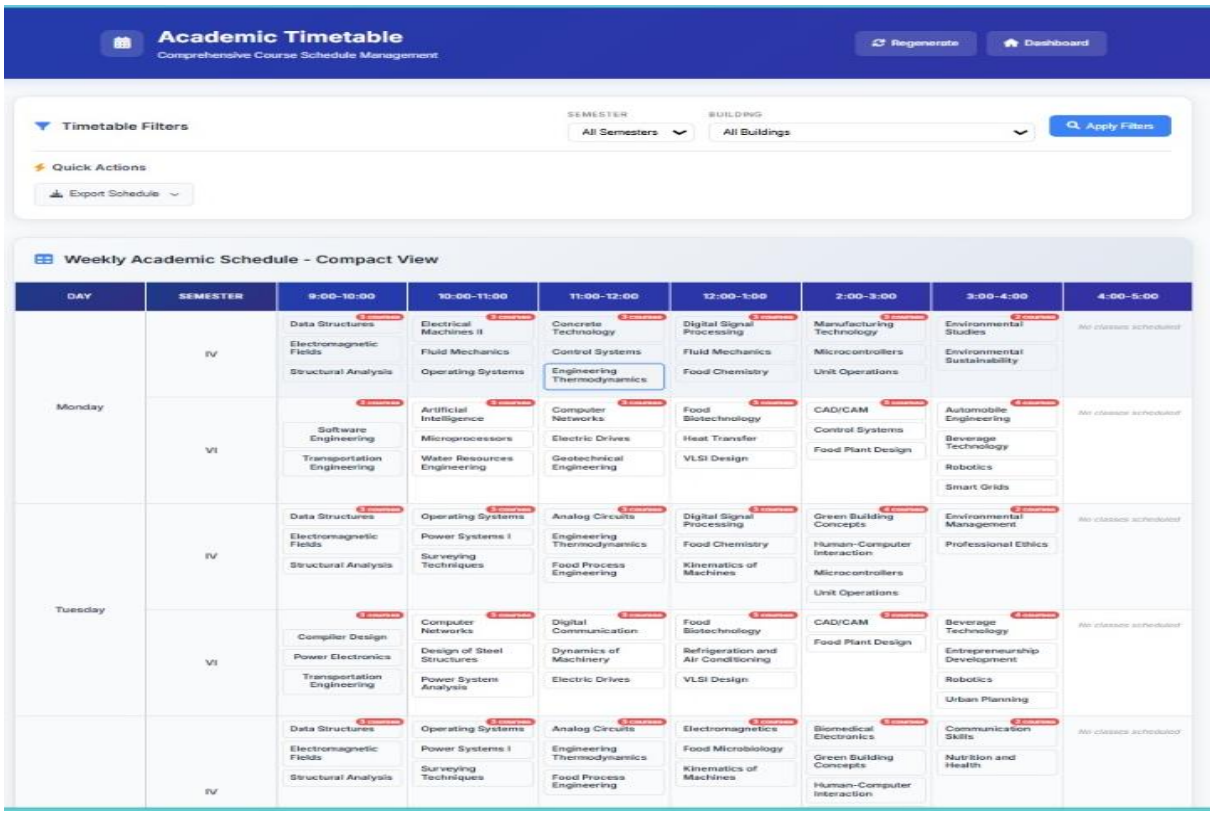


Fig 5.6.2 Course Management page (Adding, Updating, Deleting & Viewing Course)

5.6.3 Timetable View



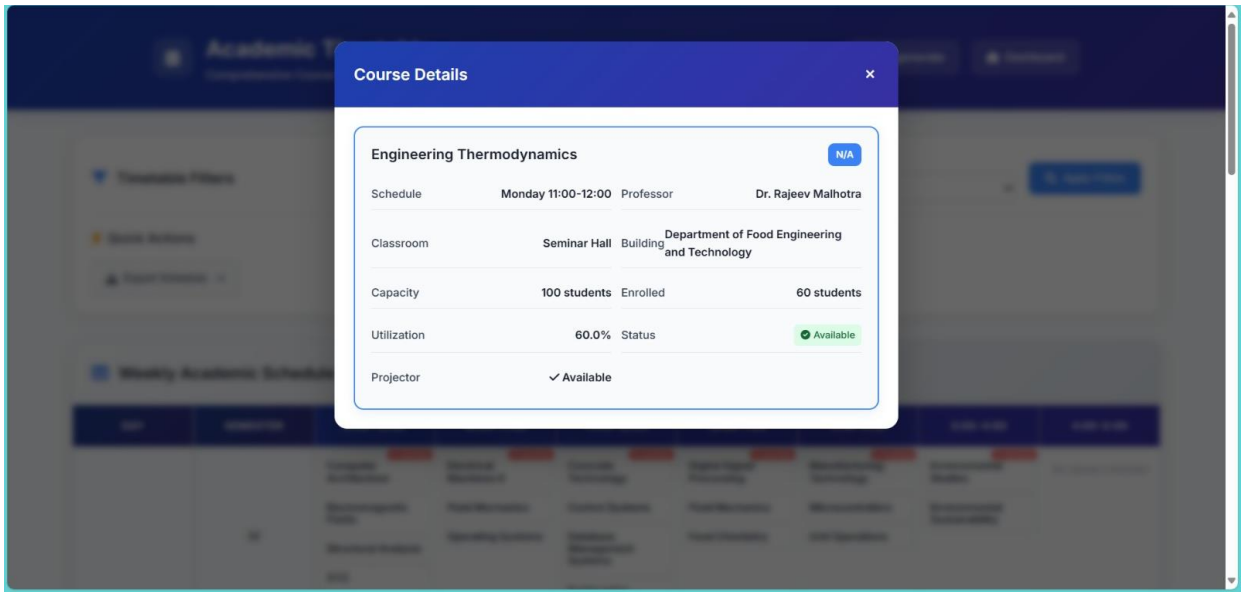


Fig 5.6.3 Final generated Academic Timetable page

5.6.4 Professor Portal

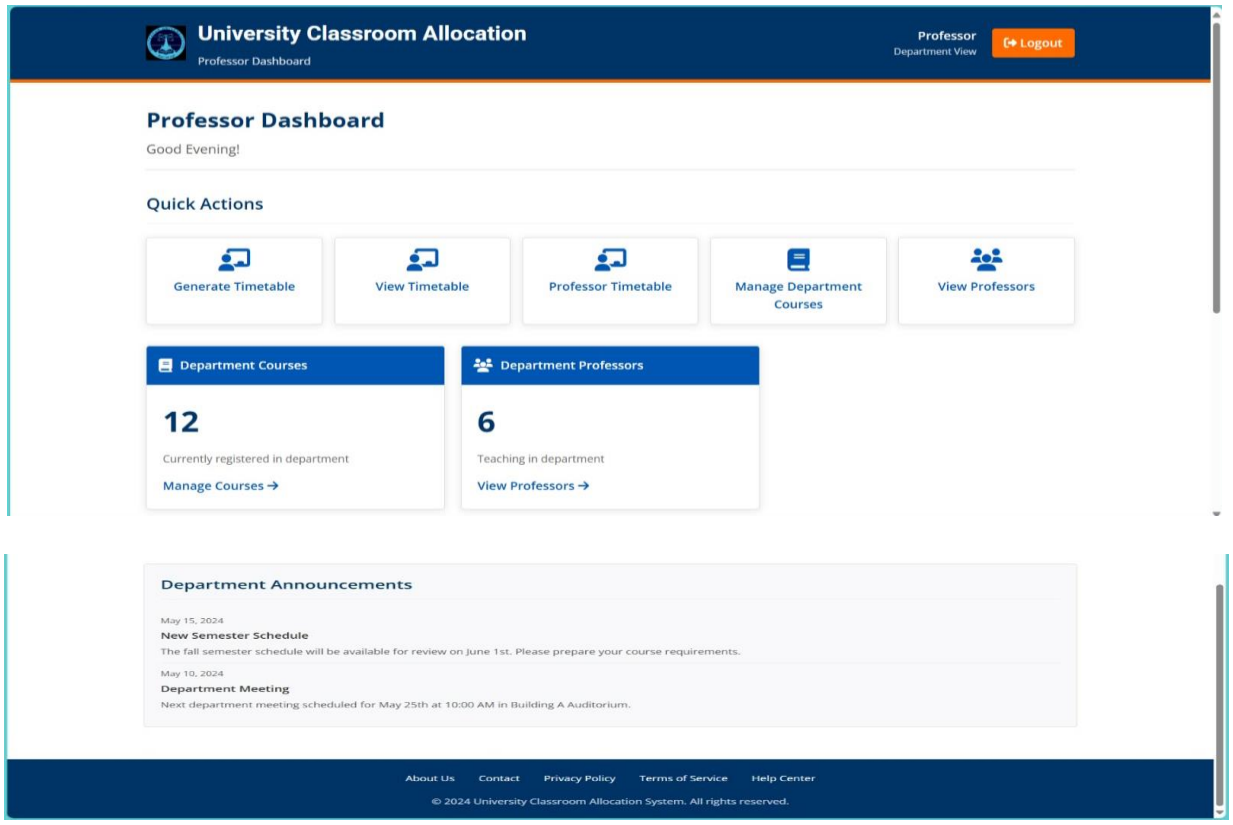


Fig 5.6.4 Professor Dashboard page

5.6.5 Excel Upload Page

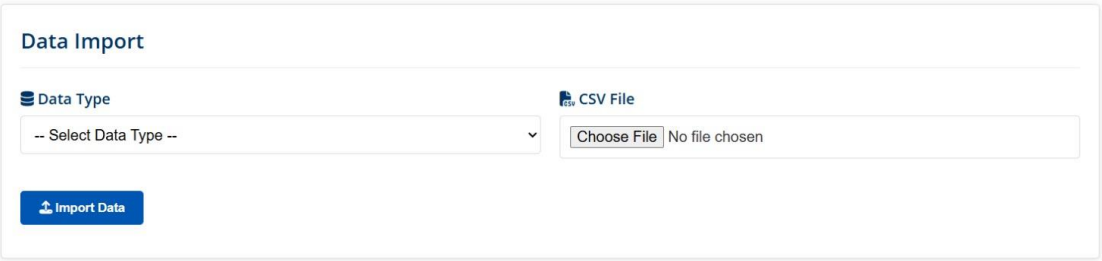


Fig 5.6.5 Excel Upload Page

5.6.6 PDF Export Button

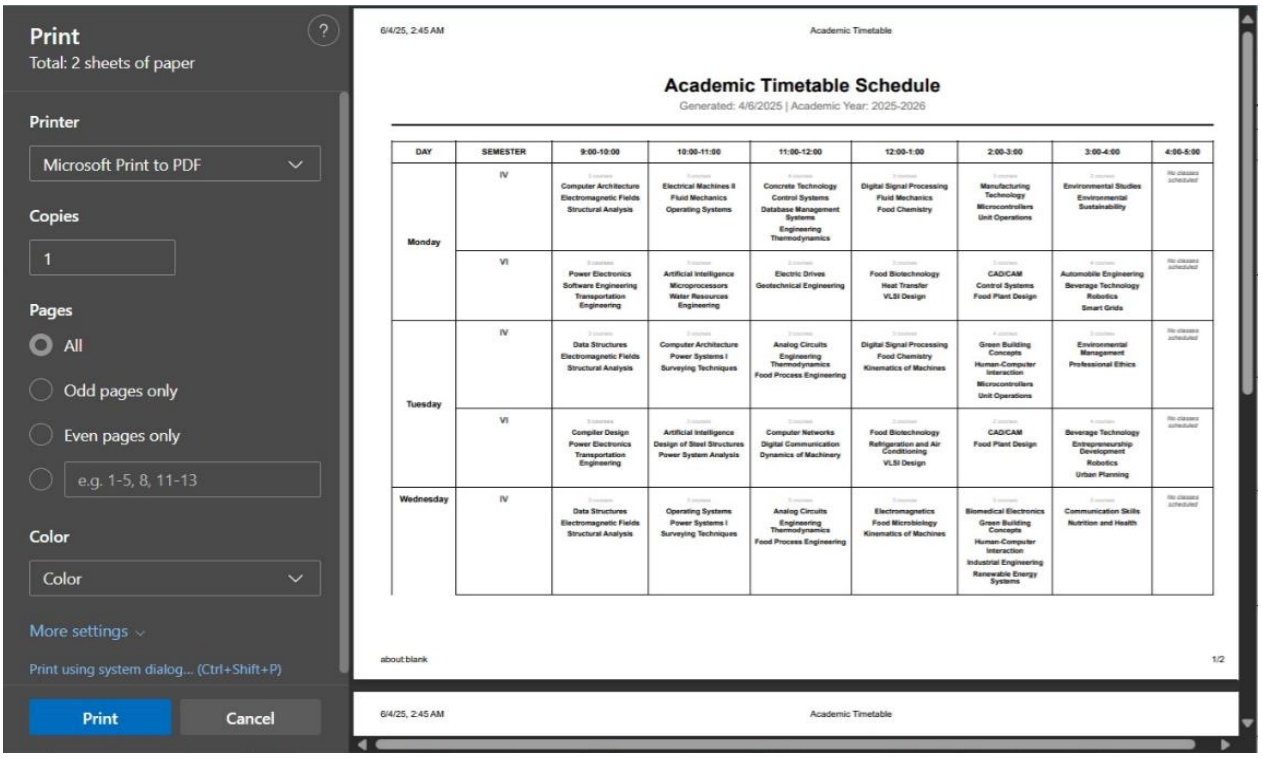


Fig 5.6.6 PDF Export Button

5.6.7 Professor Timetable

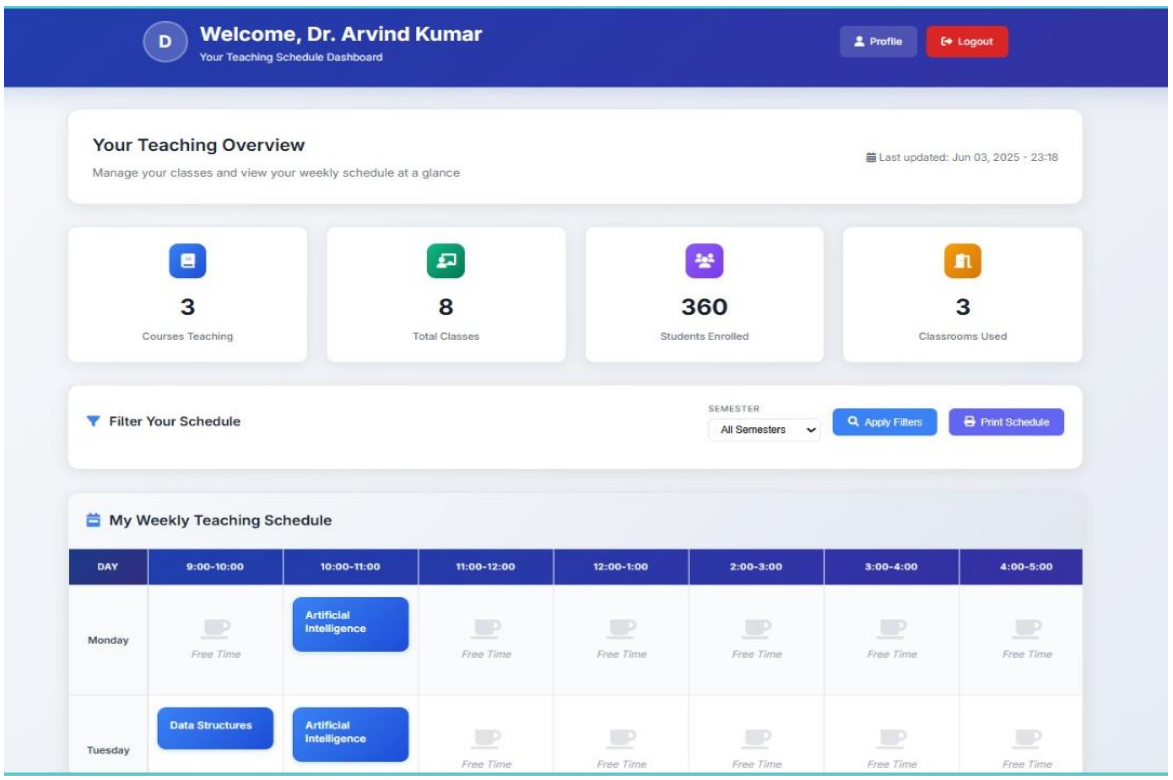


Fig 5.6.7 Professor Timetable

CHAPTER 6: UNIT TESTING AND RESULTS

6.1 Testing Strategies

To ensure system stability, correctness, and usability, multiple levels of testing were performed throughout the development cycle. These testing phases helped identify and fix potential issues before deployment.

6.1.1 Unit Testing

Unit testing was carried out for individual modules and functions to verify that they performed as expected in isolation. Some of the tested units include:

- **Form Validation Logic**: Ensuring that data input (like course credits, classroom capacity) meets expected formats.
- **Database Operations**: Insertion, retrieval, and deletion of records in tables like courses, professors, and classrooms.
- **Excel Upload Parser**: Parsing `.csv` files for bulk data upload and verifying accuracy in stored records.
- **PDF Export Module**: Ensuring timetable views are correctly rendered into a PDF format.
- **Slot Allocation Logic**: Each course's sessions are correctly scheduled according to L+T hours.

Tools Used: Manual testing using echo and print_r() for real-time verification during development.

6.1.2 Integration Testing

Once unit tests were successful, modules were integrated and tested together to ensure their interactions worked properly:

- **Admin Operations**: Adding or updating data via forms reflected in the database correctly.
- **Session Management**: Users are redirected correctly based on login roles (admin or professor).
- **Excel Upload → Timetable Generation**: Verifying that data upload correctly triggers timetable generation.
- **PDF Output**: Exported timetable accurately reflects scheduled data.

6.1.3 System Testing

System testing was performed as a complete user simulation:

- **Multi-role Testing**: Logging in with both admin and professor credentials.
- **Workflow Simulation**: Full flow from data entry to timetable generation was tested.

- **Constraint Verification**: Conflicts such as overlapping classes, invalid room assignments were successfully avoided.

Result: The system handled all major functionalities correctly and smoothly.

6.1.4 User Acceptance Testing (UAT)

Final testing was done by the development team and peers to ensure real-world usability:

- **Interface Usability**: Layouts, forms, and dashboards were intuitive and easy to navigate.
- **Correct Output**: Timetable generation adhered to course constraints.
- **Access Management**: Admin and professors only accessed their designated modules.
- **Bulk Import/Export**: Uploading `.csv` files and downloading PDFs worked without issues.

6.2 Test Cases and Results

Test Case	Input	Expected Result	Actual Result	Status
Login as Admin	Valid admin credentials	Redirect to Admin Dashboard	Passed	☑
Add Classroom	Capacity = 60, Projector = Yes	Classroom added to database	Passed	☑
Assign Course	L=3, T=1, Sem=I, Prof=Assigned	4 sessions scheduled	Passed	☑
Generate Timetable	Courses from Sem-I and Sem-II	Conflict-free schedule	Passed	☑
Excel Import	Valid .csv file of 100+ course records	Data correctly inserted into tables	Passed	☑
PDF Export	Finalized timetable view	Downloadable PDF file created	Passed	☑
Manual Slot Assignment	Select Course: DBMS, Day: Monday, Time: 10–11 AM	Session added to timetable manually	Passed	☑

6.3 Performance Evaluation

The system was benchmarked for performance under standard operating conditions (Windows 10, XAMPP, Chrome browser). The following metrics were observed:

Metric	Observation
Timetable Generation Time	< 3 seconds for over 70 courses
Excel Import Speed	~5 seconds for 100+ records
PDF Export Speed	Timetable PDF rendered in 1–2 seconds
Conflict Detection Accuracy	All hard constraints successfully enforced

Conclusion: The system performs efficiently even with large datasets, making it suitable for real-world academic use.

6.4 Limitations Observed

While the system achieves its core objective of automated classroom and timetable allocation, a few limitations remain:

- **No Student Enrollment Handling**: The system does not manage per-student registrations or backlog courses.
- **Fixed Semester-Course Mapping**: Courses are hard-assigned to specific semesters, which may not suit electives or repeated subjects.
- **No ERP/University API Integration**: The platform operates in isolation and is not yet integrated with centralized university management systems.
- **Open Elective Clashes**: Cross-department elective conflicts are not automatically resolved.

CHAPTER 7: CONCLUSION AND FUTURE WORK

7.1 Conclusion

The *University Classroom Allocation System* was successfully developed as a web-based application aimed at automating and streamlining the scheduling of academic classrooms and timetables. The system significantly reduces the manual burden traditionally associated with classroom allocation, addressing key issues such as overlapping sessions, underutilized classrooms, and scheduling conflicts involving faculty members.

At its core, the system employs a **Greedy Algorithm** that auto-generates timetables while strictly adhering to hard constraints, such as:

- Professor availability
- Room capacity and availability
- Course credit hours and lecture-tutorial combinations

In addition, soft constraints like projector requirements and preferred room attributes are considered to enhance scheduling quality.

A notable feature of the system is its **manual scheduling override**, which empowers administrators to make adjustments or handle special scenarios—an essential element for real-world adaptability.

To ensure ease of data handling and operational efficiency, the system supports **Excel-based bulk uploads** and **PDF exports**. Its intuitive, role-based interface accommodates both administrators and professors with dedicated dashboards, offering a streamlined user experience.

The successful implementation of this system demonstrates its practicality for institutional deployment and its potential to greatly reduce administrative workload, while improving accuracy and transparency in classroom scheduling.

7.2 Summary of Contributions

This project delivers several key innovations and practical solutions:

- **Complete Web-Based Scheduling Platform**: Customized for university-level classroom and timetable allocation.
- **Greedy Algorithm Implementation**: Ensures quick and conflict-free scheduling decisions.
- **Hard and Soft Constraint Handling**: Satisfies essential academic requirements while optimizing quality.
- **Manual Scheduling Support**: Provides flexibility for special adjustments and administrative overrides.
- **Role-Based Access Control**: Different views and functionalities for admin and faculty users.

- **Bulk Excel Upload**: Significantly speeds up data entry and minimizes manual errors.
- **PDF Export Feature**: Allows timetables to be downloaded and distributed for offline access.
- **Structured Database Design**: A normalized MySQL schema supporting scalability and efficient data management.
- **Improved Efficiency**: Reduces human errors and administrative workload in academic planning.

7.3 Future Work

Although the system achieves its primary goals, there is room for expansion and improvement. Future developments could include:

- **Student Course Enrollment Integration**
  - Allow students to select their courses and auto-generate personalized timetables.
- **Support for Backlog and Repeating Students**
  - Extend scheduling logic to accommodate students repeating courses from previous semesters.
- **Cross-Departmental Elective Conflict Resolution**
  - Dynamically detect and resolve scheduling conflicts in open or interdisciplinary electives.
- **Advanced Scheduling Algorithms**
  - Introduce AI-based optimization techniques like Genetic Algorithms or Simulated Annealing for improved soft constraint satisfaction.
- **Graphical Drag-and-Drop Scheduling Interface**
  - Provide an interactive UI for administrators to manually modify and visualize timetables.
- **University ERP Integration**
  - Connect with institutional academic portals for real-time data syncing (e.g., student info, course lists).
- **Mobile Application**
  - Develop a mobile-friendly app for students and faculty to access and download their timetables.
- **Notification System**
  - Implement email or SMS alerts for notifying students and professors of timetable updates or changes.

REFERENCES

[1] Boomija, M. D., Ambika, R., Sandhiya, J., Jayashree, P. (2019) Smart and Dynamic Timetable Generator, International Journal for Research in Applied Science and Engineering Technology. https://ijarsct.co.in/A9370.pdf

[2] Abhinaya, V., Sahithi, K., Akaanksha, K. (2019) Online Application of University Timetable generator, International Research Journal of Engineering and Technology. https://ijcrt.org/papers/IJCRT2304429.pdf

[3] Puttaswamy, A., Khan, A. A., Chandan, S. V, Parkavi., A. (2019) A Study on University Timetable Generator, International Journal of Science and Innovative Engineering 10.23956/ijarcsseSV7I5/0234

[4] Pai, A. R., Ashwitha, S., Shetty, R., Laxmi, G. (2018) University college timetable generator, International Journal of Scientific & Engineering Research, vol. 9, no. 4. https://www.ijser.org/researchpaper/University-college-timetable- generator.pdf

[5] Saritha, M., Pranav., K. V., Pradeep, Mahesh, N. R. (2017) University time table generator, International Journal of Advanced Research in Computer Science and Software Engineering. DOI: 10.23956/ijarcsse/SV7I5/0234

[6] Mittal, D., Doshi, H., Mohammed, S., Nagpure, R. (2015) University Timetable Generator using Genetic Algorithm, International Journal of Advanced Research in Computer and Communication Engineering. https://ijsart.com/Content/PDFDocuments/IJSARTV8I554760.pdf

[7] Chowdhary, A., Kakde, P., Dhoke, S., Ingle, S., Rushiya, R., Gawande, D. (2014) Timetable Generation System, IJCSMC, vol. 3, no. 2. https://www.irjmets.com/uploadedfiles/paper/issue_4_april_2023/35814/final/fin_irjmets1681375100.pdf

[8] Sadaf, N. J., Shengxiang, Y. (2008) "A mimetic algorithm for university course timetabling problem", 2008 20th IEEE International conference on tools with artificial intelligence.https://bura.brunel.ac.uk/bitstream/2438/5842/2/Fulltext.pdf

[9] P. Kumar Chaki and S. Anirban, "Algorithm for efficient seating plan for centralized exam system", International Conference on Computational Techniques in Information and Communication Technologies (ICCTICT),2016. https://www.researchgate.net/publication/305421470_Algorithm_for_efficient_seating_plan_for_centralized_exam_system

[10] Tiwari, S., Lal, A., Agarwal, S., Kumar, A., Singh, A., & Arora, N. Novel bit-level Adaptive and Asymmetric Data Compression Technique. International Journal of Computer Applications, 975, 8887.https://www.researchgate.net/publication/339335716_Novel_bitlevel_Adaptive_and_Asymmetric_Data_Compression_Technique